HH-SignalProcessing: what is on `gabriel` and how to run it

Acoustic drone detector with 360° haptic bearing for the headband · 2026-10-08 · branch `gabriel` (head `bd60f4b`) · for the hackathon team

1. Summary

What it is. A headband that hears a multirotor drone and tells the wearer where it is by touch. Three Adafruit PDM MEMS mics (ST MP34DT01-M) on an ESP32-S3 sample sound in sync. A harmonic-comb detector and a small CNN decide whether a drone is present, and masked GCC-PHAT across the three mics gives a 360° bearing. Three coin motors (Left, Right, Back) vibrate toward the drone.

Status	Item
Done, runs on a Mac	NumPy reference pipeline (`droneears/`): FFT front end, log-mel, harmonic detector, 3-mic bearing, fusion, haptic panning. Scripts for file and live-mic runs, the ESC-50 false-alarm eval, training and export.
Done, verified on laptop	Portable C++ core (`firmware/core/`). On a synthetic 3-channel clip it matches the Python frame by frame (f0, score, flatness, hits, state and bearing all agree; max bearing difference 0.0005°; `scripts/compare_core.py`). 28 pytest tests in `tests/`.
Done, trained	int8 TFLite classifier: 15 698 parameters, 24 320 bytes, 0.06 ms per window on a laptop. Exported as a C array (`model_data.h`).
Written, untested	ESP32-S3 sketch (`firmware/esp32s3/headband.ino`): PDM capture on I2S0, DSP on core 0, classifier, haptics and telemetry on core 1. **It has never been compiled or run on hardware.**
Not done	Hardware bring-up (bench checks, section 7). A field session through the real headband mics, a retrain on it, and threshold tuning on a held-out session (section 8).

Measured so far (details and caveats in section 6)

- **Real phone clips:** the fused detector (rules AND classifier) reaches DETECTED on all 9 committed drone recordings, first detection at 2.6–9.3 s. It never fires on the 237 s ambient recording. *The thresholds were tuned on these same 9 clips*, so 9/9 is optimistic.
- **False alarms, ESC-50 (all 2000 clips):** fused 0.19 % of frames / 11 clips; rules alone 2.2 % of frames / 5.6 % of clips.
- **Classifier (int8, held-out split, p > 0.5):** accuracy 94.9 %, recall 88.2 %, false-positive rate (FPR) 3.2 %.
- **Bearing (simulation only):** 0.3–0.6° median error at 10 dB SNR. A drone circling the head over real ambient at 3 dB SNR is tracked with 4.3° median error.

Quick start

```
brew install ffmpeg
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python -m pytest -q tests          # 28 tests
.venv/bin/python scripts/demo_direction.py   # drone circles the head ->
artifacts/direction_demo.png
.venv/bin/python scripts/run_file.py "richard_data/drone/Fly by 1.m4a"
```

2. Architecture: the per-frame pipeline

One frame is 2048 samples at 48 kHz (42.7 ms, 23.44 Hz bins), with a new frame every 1024 samples (21.3 ms, 46.9 frames/s). The Python (`droneears/pipeline.py`), the C++ core and the sketch all run the same loop.

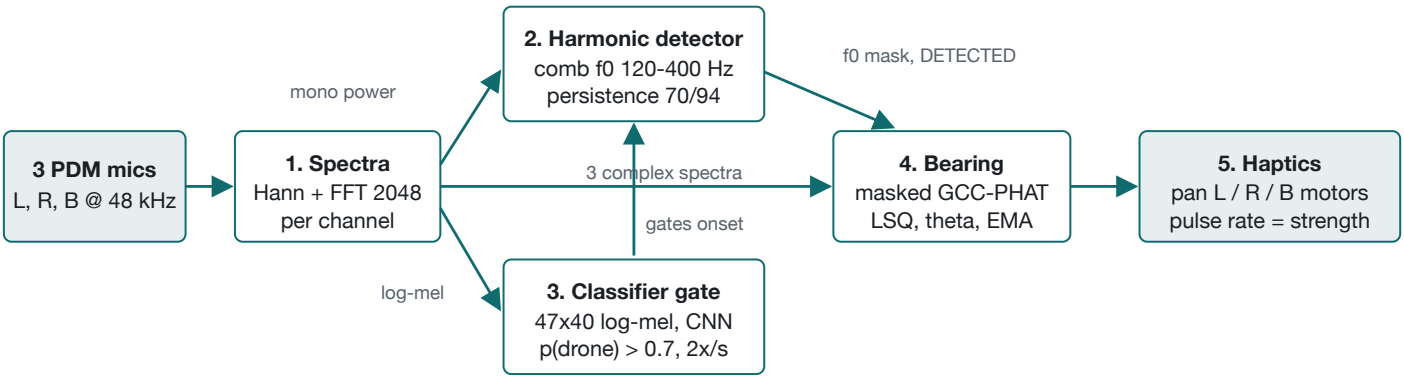


Figure 1. Per-frame data flow. The classifier only gates the onset of a detection and can never trigger one on its own. The bearing uses only the bins at multiples of the detected f0.

Stage (file)	What it does, with the numbers from `README.md` / `droneears/config.py`
1. Spectra `features.py`	Each channel gets a periodic Hann window and an FFT. The mono power spectrum feeds the detector and the classifier; the three complex spectra feed the bearing. One FFT per channel per frame is the whole compute budget.
2. Harmonic detector `detector.py`	Prominence = the dB spectrum minus a smooth baseline that ignores peaks (31 bins, about 730 Hz), averaged over frames with an EMA (0.5 weight on the new frame). The comb tries every f0 from 120 to 400 Hz in 2 Hz steps and scores the mean prominence of 5 consecutive harmonics, starting at the first one at or above 300 Hz. Hysteresis: a candidate within 6 % of the previous f0 is kept if it scores within 0.5 dB of the best. A frame counts as drone evidence when all of these hold: • score > 5.5 dB • at least 3 of the 8 harmonics are > 6 dB proud • spectral flatness in 300–3000 Hz is between 0.17 and 0.45 (this rejects pure tones and white noise) • f0 is steady: at least 35 % of the last ~1 s of estimates are within 6 % of their median, checked once 0.5 s of history exists **DETECTED** needs 70 of the last 94 frames (~1.5 s of evidence in a 2 s window). It releases after 2 s with ≤ 20 frames.
3. Classifier gate `classifier.py`, `scripts/train.py`	Input: a 47×40 log-mel window (1.0 s, 40 bands from 100 to 8000 Hz), normalised to the window maximum and clipped to −80 dB. Model: a CNN with 4 conv layers, global pool and dense (15 698 parameters, 24 KB int8), run twice a second. Fusion: a detection can only *start* if p > 0.7 on the latest update or on any update in the last ~1 s (47 frames).
4. Bearing `bearing.py`	For each mic pair: cross-spectrum $\mathrm{conj}(X_i)\cdot X_j$, PHAT-normalised and masked to the bins at k·f0 (8 harmonics, ±1 bin), then an inverse FFT and a parabolic peak fit give the delay τ. Least squares turns the three delays into a direction: $u = A^{+}\tau$. $\lvert u\rvert \approx 1$ is a free consistency check, with a minimum of 0.6. $\theta = \mathrm{atan2}(u_x, u_y)$: 0° ahead, +90° right, ±180° behind. A circular EMA (α = 0.1) smooths θ while DETECTED. Mics: L (−0.10, 0), R (+0.10, 0), B (0, −0.12) m.
5. Haptics `bearing.py`, sketch	The two motors bracketing θ share the vibration (L −90°, R +90°, B 180°); the pulse rate follows detection strength. Drone to the left buzzes L; back-left buzzes B and L; straight ahead buzzes L and R together (there is no front motor). **Overhead** buzzes all three at full weight. The mics are coplanar, so conf = $\lvert u\rvert \approx \cos(\text{elevation})$: overhead is declared when conf < 0.5 for 23 consecutive frames (~0.5 s) while DETECTED, and released when conf > 0.7 for 23 frames. θ stays frozen while overhead.

Every threshold and the mic geometry live in `droneears/config.py`. `scripts/export_c_headers.py` regenerates `firmware/core/dsp_config.h` from it, so the C core cannot drift from the Python.

3. Repository map

Branch `gabriel` vs `main`: 44 files changed, +6968 / –226 lines (`git diff --stat main...HEAD`), in two commits: `718149f` (pipeline, bearing, classifier, firmware) and `bd60f4b` (overhead haptics, serial protocol, host tools for device bring-up). Older history: `e9b6a8b` triangulation proof of concept, `7dfe19f` first model, `5c7462b` / `9bab931` Richard's recordings and ambient data.

Path	Purpose
`README.md`	Overview, quick start, pipeline description with measured numbers, porting status, ESP32 memory notes.
`TRAINING_PLAN.md`	Classifier data sources, splits, augmentation, current metrics, fine-tune procedure after the field session.
`droneears/config.py`	Every shared constant: FS, FFT, thresholds, mic geometry, motor angles. Source of `dsp_config.h`.
`droneears/audio.py`	Loads any audio file through ffmpeg and resamples it to 48 kHz.
`droneears/features.py`	Framing, Hann, FFT, log-mel, window normalisation.
`droneears/detector.py`	Noise floor, harmonic-comb search, detection state machine, bearing masks.
`droneears/bearing.py`	Masked GCC-PHAT, least-squares direction, `BearingEstimator`, `haptic_weights`.
`droneears/pipeline.py`	`Pipeline.run()`: the frame loop (spectra → detector → classifier → bearing → haptics); `summarise()`.
`droneears/classifier.py`	Loads the TFLite model as a callable (uses `ai_edge_litert` if installed, else `tf.lite`).
`droneears/dataset.py`	Builds train/test windows from team clips, ESC-50, DADS and synthetic clips. Splits by source and handles `*_holdout` files.
`droneears/simulate.py`	Synthetic quadcopter and 3-mic arrival simulation (static and moving sources).
`scripts/*.py`	Runnable programs (section 5). Laptop-only: `demo_direction`, `run_file`, `live_mac`, `eval_esc50`, `get_dads`, `train`, `export_c_headers`, `compare_core`, `retune_holdout`. Device (serial): `telemetry`, `record_device`.
`tests/test_pipeline.py`	pytest suite (section 5.1).
`firmware/core/dsp_core.{h,cpp}`	Portable C++ core: no heap after init, no dependencies beyond `<math.h>`.
`firmware/core/dsp_config.h`	Auto-generated from `config.py`. Do not edit by hand.
`firmware/core/test_core.cpp`	Laptop harness: reads a 3-channel WAV and prints one CSV row per frame (used by `compare_core.py`).
`firmware/esp32s3/headband.ino`	ESP32-S3 sketch. `src/` holds symlinks to `../core`, because Arduino only compiles files inside the sketch folder.
`firmware/esp32s3/model_data.h`	Copy of `artifacts/model_data.h` (int8 model as a C array) that the sketch includes when `USE_TFLM 1`.
`artifacts/`	`drone_cls.keras`, `drone_cls_float.tflite` (67 844 B), `drone_cls_int8.tflite` (24 320 B), `model_data.h`, `train_report.json`, `direction_demo.png`. Also `example_drone_3ch.wav` and `probs8_test.npy`, which no script references. `features_test.npz` and `compare_input.wav` are generated and git-ignored.
`richard_data/drone/`, `richard_data/ambient/`	Committed phone recordings (m4a): 9 drone clips (~96 s) and `ambient_adam.m4a` (237 s). Used by the tests and by training.
`data/team/{drone,background}/`	Put new recordings here (not committed; see `data/team/README.md`).
`data/ESC-50/`, `data/hf-drone/`	External datasets (git-ignored): the ESC-50 clone and the DADS parquet shards.
`detection_model.py`, `sig_proc_tri.py`	Proof-of-concept scripts from `main`, left untouched (PyTorch CNN on DADS; triangulation). They need torch, datasets and scikit-learn, which `requirements.txt` does not include.

4. Setup on a Mac

1. **ffmpeg** (decodes the m4a/mp3 recordings): `brew install ffmpeg`

2. **Virtual environment and requirements**:

```
cd HH-SignalProcessing
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

`requirements.txt`: numpy, scipy, soundfile, matplotlib, tensorflow ≥ 2.16 (training and TFLite), sounddevice (`live_mac.py` only), pytest, pyserial ≥ 3.5 (device scripts), pyarrow (DADS parquet). The examples below call `.venv/bin/python` directly; `source .venv/bin/activate` and then `python` works just as well.

3. **ESC-50** (needed by `eval_esc50.py` and `train.py`; optional otherwise):

```
git clone --depth 1 https://github.com/karolpiczak/ESC-50 data/ESC-50     # or set ESC50_DIR
```

4. **DADS public drone audio** (optional; training skips it if it is missing):

```
.venv/bin/python scripts/get_dads.py                                  # default: shards 0,38
.venv/bin/python scripts/get_dads.py --shards 0,5,12,20,28,36         # the 6 shards (~1.7 GB)
behind the shipped model
```

The files go to `data/hf-drone/` (or `DADS_DIR`). The highest-numbered shard present becomes the test shard.

5. **A C++ compiler** for `compare_core.py` and the C++ test: `g++` from the Xcode command-line tools (`xcode-select --install`).
6. **Check:** `.venv/bin/python -m pytest -q tests` should report 28 passed in about 20 s.

> **Run pytest on `tests`, not the whole repo.** The README says `pytest -q`. Once ESC-50 is cloned into `data/ESC-50`, a bare `pytest -q` also collects `data/ESC-50/tests/test_dataset.py`, which imports `librosa` and stops collection with an error. Use `pytest -q tests`.

5. Running each program

Run everything from the repo root. Every script adds the repo root to `sys.path`, so `droneears` needs no install. Scripts that use the classifier load `artifacts/drone_cls_int8.tflite` by default (`--model` overrides it) and print a TensorFlow deprecation warning, which is harmless.

5.1 Tests: `pytest`

```
.venv/bin/python -m pytest -q tests      # 28 passed, ~20 s
```

Test	Checks
`test_bearing_all_quadrants`	Simulated source recovered in all compass directions, including behind.
`test_overhead_drone_buzzes_all_motors`	Identical signal on all mics (a drone straight above): detected, and overhead declared within 1 s of detection, so all motors buzz.
`test_detects_within_two_seconds`, `test_white_noise_does_not_detect`	Detector latency on a synthetic drone; white noise does not trigger.
`test_least_squares_recovers_unit_vector`	The delay → direction solve.
`test_haptic_weights_sum_to_one_and_pan`	Motor pan weights.
`test_mel_filterbank_rows_normalised`	Mel filterbank.
`test_cpp_core_matches_python`	Builds the C++ core with g++ and compares it with the Python.
`test_real_drone_recording_detected[…]` ×9, `…_fused[…]` ×9	Each of the 9 `richard_data/drone` clips reaches DETECTED, rules only and fused.
`test_real_ambient_recording_not_detected`, `…_fused`	The 237 s ambient recording never reaches DETECTED.

5.2 `scripts/demo_direction.py`: bearing demo

Purpose Simulates a drone circling the head once (clockwise, starting ahead) on three mics, runs the pipeline and plots true vs estimated bearing. Without `--drone` it uses a synthetic quadcopter; without `--bg` the ambient is white noise.

Options `--drone FILE`, `--bg FILE`, `--snr` (dB, default 6), `--seconds` (default 16), `--out` (default `artifacts/direction_demo.png`).

```
.venv/bin/python scripts/demo_direction.py
.venv/bin/python scripts/demo_direction.py --bg richard_data/ambient/ambient_adam.m4a --snr 3
# output (default run, measured today):
frames 749, detected 86%, first detection at 2.33 s
bearing error while detected: median 4.3 deg, 90th pct 4.7 deg
wrote .../artifacts/direction_demo.png
```

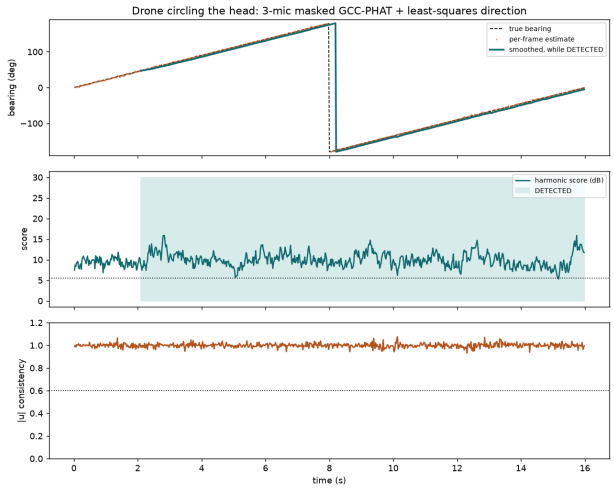


Figure 2. `artifacts/direction_demo.png` (committed). Top: true bearing (dashed), per-frame estimates (dots) and the smoothed bearing while DETECTED. Middle: harmonic score against the 5.5 dB threshold, with DETECTED shaded. Bottom: |u| consistency against the 0.6 minimum. Simulation only.

5.3 `scripts/run_file.py`: pipeline over a recording

Purpose Prints a timeline of the detector and classifier for any file ffmpeg can read. Mono files get detection only. 3-channel files (L, R, B) also get the bearing. Other channel counts use channel 0 only; the phone m4a files are 2-channel.

Options `path`, `--no-ml` (rules only), `--plot out.png` (score, ml×20 and DETECTED; plus bearing for 3-channel files), `--model PATH`.

```
.venv/bin/python scripts/run_file.py "richard_data/drone/Fly by 1.m4a"
Fly by 1.m4a: 6.5 s, 1 ch, classifier on
detected 60% of frames; first detection 2.6026666666666665 s
  t(s)   f0(Hz) score  flat  harm  hits  ml    state
   2.5     264   7.7  0.25    6    68  0.99    -
   2.9     234  12.5  0.20    7    86  1.00  DRONE        (one row per ~0.5 s)
```

Columns: f0 estimate, comb score (dB), spectral flatness, proud harmonics (of 8), persistence hits (70 needed), classifier p(drone), state. 3-channel files add `theta` and `conf`. In section 8 this is the tool for reading score distributions.

5.4 `scripts/live_mac.py`: live on the laptop mic

Purpose Runs the detector and classifier live, printing a status line about every 0.25 s. With the built-in mic it gives detection only. The bearing needs `--device` pointing at an interface with 3 or more inputs, in channel order L, R, B.

Options `--list` (show audio devices), `--device N`, `--no-ml`, `--model PATH`. Ctrl-C stops it. macOS asks for microphone permission the first time.

```
.venv/bin/python scripts/live_mac.py
.venv/bin/python scripts/live_mac.py --list
.venv/bin/python scripts/live_mac.py --device 3
```

Only `--help` was run while writing this document; the live mode was not exercised.

5.5 `scripts/eval_esc50.py`: false-alarm rate

Purpose ESC-50 contains no drones, so every DETECTED frame is a false alarm. Reports (a) the fraction of frames DETECTED, (b) the fraction of clips with any detection and (c) a per-

category table, sorted by frame rate.

Options `--no-ml` (rules only) or `--model PATH`; `--per-class N` (default all 40 per category) with `--seed`; `--folds 1,2,3,4,5`; `--esc50 DIR`; `--json out.json`.

```
.venv/bin/python scripts/eval_esc50.py                      # fused, all 2000 clips
.venv/bin/python scripts/eval_esc50.py --no-ml              # rules only
.venv/bin/python scripts/eval_esc50.py --per-class 5 --json /tmp/esc.json   # quick subset
```

5.6 `scripts/get_dads.py`: download DADS

Purpose Downloads parquet shards of DADS (Drone Audio Detection Samples, Hugging Face, MIT licence; 39 shards of about 175 MB, 16 kHz mono). Shards already present are skipped.

Options `--shards` (comma-separated indices 0–38, default `0,38`), `--out` (default `data/hf-drone`).

```
.venv/bin/python scripts/get_dads.py --shards 0,5,12,20,28,36
```

The shards are sorted by label: shard 0 is no-drone (1.27 GB locally) and the rest are drone. The shipped model used the six shards above.

5.7 `scripts/train.py`: train and export the classifier

Purpose Builds windows from `richard_data/`, `data/team/*`, DADS, ESC-50 and synthetic clips. Trains the CNN and reports float and int8 accuracy, recall and FPR on the held-out split, plus the worst groups. Then exports the models. Takes about 4 minutes on a laptop CPU.

Flag	Default	Meaning
`--epochs`	25	Maximum epochs; early stopping on val_loss (the shipped model stopped at 18).
`--n-synth`	120	Synthetic quadcopter clips. The shipped model used 60; the next retrain should try 0–30.
`--n-dads-pos`	200	DADS 2.5 s drone segments for training (test uses 1/4); 0 disables.
`--n-dads-neg`	100	DADS no-drone clips for training (test uses 1/4); 0 disables.
`--esc-per-fold`	all	Limit ESC-50 clips per fold.
`--out`, `--seed`	artifacts, 0	Output directory; random seed.

```
.venv/bin/python scripts/train.py --epochs 30 --n-synth 60     # command behind the shipped model
```

Writes to `--out`: `drone_cls.keras`, `drone_cls_float.tflite`, `drone_cls_int8.tflite`, `model_data.h` (C array plus `MODEL_IN/OUT_SCALE/ZERO`), `train_report.json` (window counts, float/int8 acc, recall, FPR, TP/FN/FP/TN, sizes, parameters, epochs, quantisation) and `features_test.npz`. **This overwrites the committed model.** To try something without replacing it, pass `--out /tmp/try`. Afterwards, copy `model_data.h` into `firmware/esp32s3/` by hand.

5.8 `scripts/export_c_headers.py`: regenerate `dsp_config.h`

Writes `firmware/core/dsp_config.h` from `droneears/config.py`: all constants, the mic positions, TAU_MAX and the least-squares pseudo-inverse. It has no options: even `--help` just runs it and prints `wrote .../dsp_config.h`. If the config has not changed, the output is identical. Run it after every edit to `config.py`; `compare_core.py` also runs it first.

```
.venv/bin/python scripts/export_c_headers.py
```

5.9 `scripts/compare_core.py`: C++ core vs Python

Purpose Regenerates the header, builds `firmware/core/test_core` with `g++ -O2 -std=c++11`, runs it and `Pipeline` on the same 3-channel WAV and compares f0, score, flatness, hits, detected, conf, theta_raw and theta frame by frame. Exits 1 on a mismatch.

Options `--wav FILE` (3-channel). The default is a synthetic drone at +60° with noise.

```
.venv/bin/python scripts/compare_core.py
frames: C 186  Python 186
  theta      max|diff|    0.0005   frames over tol   0.0%   (tol 1.5)
  C theta while detected: median 60.4 deg;  Python: 60.4 deg
CORE MATCHES PYTHON
```

5.10 `scripts/telemetry.py`: live monitor and logger (device)

Purpose Connects to the headband, sends `T` and shows one status line live: state, f0, score, hits, ml, theta, conf, bars for the L/R/B motor weights, and frames/s. It can log every `T` line to CSV (with a `host_s` column prepended) and plot a log offline. Columns are parsed by header name, and the parser resyncs after malformed lines.

Options `--port` (default: auto-detect), `--list-ports`, `--log FILE.csv`, `--seconds N` (default: until Ctrl-C), `--plot LOG.csv` with `--out PNG` (offline 4-panel figure, no device needed).

```
.venv/bin/python scripts/telemetry.py --list-ports
.venv/bin/python scripts/telemetry.py --log run1.csv     # expect ~47 fr/s
.venv/bin/python scripts/telemetry.py --plot run1.csv --out run1.png
```

5.11 `scripts/record_device.py`: record 3-channel audio from the device

Purpose Sends `R`, parses the raw packets (resyncing on the magic bytes; gaps are zero-filled and counted) and writes a 48 kHz, 16-bit, 3-channel (L, R, B) WAV: exactly what the DSP sees. Every second it prints per-channel RMS and peak (dBFS) and DC offset (% of full scale). On exit it sends `T` and prints a summary with a clipping/DC verdict. Use the native USB port (`/dev/cu.usbmodem*`).

Options `--seconds` (default 60); `--out FILE.wav`, or the default `data/team/background/device_<timestamp>.wav` (`data/team/drone/` with `--drone`); `--holdout` (appends `_holdout` to the name); `--monitor` (levels only, no file); `--mono-check` (afterwards, runs the rules-only pipeline on the file and prints the detection summary, max hits and median bearing); `--port`, `--list-ports`.

```
.venv/bin/python scripts/record_device.py --monitor            # tap test, DC check
.venv/bin/python scripts/record_device.py --drone --holdout --seconds 120 --mono-check
.venv/bin/python scripts/record_device.py --holdout --seconds 300   # background
```

5.12 `scripts/retune_holdout.py`: retune thresholds on the held-out session

Purpose Sweeps a grid of detector constants over the held-out drone and background clips and an ESC-50 sample. It recommends the config that detects the most drone clips, subject to two limits: no background clip may have any detection, and ESC-50 DETECTED frames must stay ≤ `--max-esc-fa` (0.5 % fused, 3 % rules-only). Ties are broken by lower ESC false alarms, then earlier median first detection, then fewer changes from the current config. For threshold-only grids the pipeline runs once per file and the state machine is replayed per config (checked against the real pipeline). Otherwise, or with `--full`, it reruns the pipeline per config

in parallel. Constants that are consumed at import time (comb tables, frame geometry) are refused.

Options `--drone` (default `data/team/drone/*_holdout*`); `--bg` (default `data/team/background/*_holdout*` plus `richard_data/ambient/*.m4a`); `--esc-per-class` (default 8, 0 = off) with `--seed`; `--no-ml`, `--model`; `--grid NAME=v1,v2`, repeatable (default `HARM_THRESH_DB=4.5,5.5,6.5,7.5 FLAT_MIN=0.10,0.14,0.17,0.20,0.24 PERSIST_ON=60,70,80`); `--max-esc-fa`; `--jobs`; `--full`; `--top N`; `--apply` (writes the winner into `droneears/config.py` and runs `export_c_headers.py`); `--config`.

```
.venv/bin/python scripts/retune_holdout.py                    # fused, default grid
.venv/bin/python scripts/retune_holdout.py --apply
# smoke run on the committed clips (rules only, 2 configs, no ESC), measured today:
.venv/bin/python scripts/retune_holdout.py --drone richard_data/drone/*.m4a --esc-per-class 0 --no-
ml \
    --grid HARM_THRESH_DB=5.5,6.5 --grid FLAT_MIN=0.17 --grid PERSIST_ON=70
  HARM_THRESH_DB | FLAT_MIN | PERSIST_ON | drone det | med 1st s | bg frames | bg clips | ... |
hits drone-min/bg-max
*            5.5 |     0.17 |         70 |    9/9    |       3.9 |     0.00% |   0/1    | ... |
74/63
             6.5 |     0.17 |         70 |    7/9    |       4.0 |     0.00% |   0/1    | ... |
61/49
recommended: HARM_THRESH_DB = 5.5, FLAT_MIN = 0.17, PERSIST_ON = 70  -> ... (same as current
config)
```

6. Results so far

6.1 Classifier (int8, held-out split, 5 363 windows)

The held-out split is ESC-50 fold 5, the last 30 % of every team clip, DADS shard 36 plus the last 20 % of shard 0, and 20 synthetic clips. Training used 20 617 windows; early stop at epoch 18; 15 698 parameters; 24 320 bytes (`artifacts/train_report.json`, `TRAINING_PLAN.md`).

int8, test split	p > 0.5	p > 0.7	n
all: acc / recall / FPR	94.9 % / 88.2 % / 3.2 % (TP 1058 FN 142 FP 132 TN 4031)	95.0 % / 86.1 % / 2.5 %	5 363
`team:drone` recall	84 %	82 %	44
`team:drone:mix` recall	64 %	55 %	132
`team:background` FPR	0 %	0 %	143
`dads:drone` / `dads:drone:mix` recall	92 % / 87 %	90 % / 86 %	156 / 468
`dads:background` FPR	13.6 %	12.4 %	420
synthetic / synthetic:mix recall	100 % / 95 %	100 % / 95 %	100 / 300
ESC-50 fold 5 FPR	2.1 %	1.4 % (airplane, chainsaw, insects, door creaks 11–13 %)	3 600

Float model: acc 95.0 %, recall 88.6 %, FPR 3.2 %. The 13.6 % `dads:background` FPR comes from a 25-clip sample; a fresh 60-clip sample gives 0.7 %, so a few engine-like clips dominate it.

6.2 Fused detector on the real phone recordings (channel 0)

file	dur s	fused DETECTED %	first det. s	rules-only %	max hits (need 70)
Immediate vicinity	5.3	25.5	3.9	27.1	89
Hovering 5m ahead	7.9	61.6	3.0	61.6	77

Hovering 10m away	7.1	32.0	4.8	32.0	74
Fly by 1	6.5	59.7	2.6	59.7	91
Fly by 2	8.4	49.1	3.9	49.1	80
Assorted motion	17.7	49.8	8.9	49.8	86
Descending	11.5	18.7	9.3	18.7	89
Fly by 50m height 1	16.3	81.8	3.0	81.8	81
Fly by 50m height 2	15.3	70.8	4.5	70.8	89
ambient_adam (no drone)	236.6	0.0	–	0.0	63

6.3 False alarms

Set	Rules only	Fused (rules AND p > 0.7)
ESC-50, all 2000 clips	2.2 % of frames / 5.6 % of clips (vacuum cleaner, chainsaw, toilet flush, crow, engine)	0.19 % of frames / 11 clips (chainsaw 4, toilet flush 4, washing machine 1, door creaks 1, insects 1)
60 held-out DADS no-drone clips	1 clip / 0.31 % of frames	0 clips / 0.00 %
ambient_adam.m4a (237 s)	0 %	0 %

6.4 Bearing (simulation only)

- Static source, 10 dB SNR in white noise: 0.3–0.6° median error in all eight compass directions, including behind. At 0 dB: 0.8–2.2°.
- Drone circling at 22.5°/s over the real ambient mixed in at 3 dB SNR: 4.3° median error, almost all of it EMA lag (the raw per-frame median is ~1°). At 0 dB over that ambient the detector does not fire, so there is no bearing.
- The ambient is mixed with an independent offset per mic, which makes it spatially uncorrelated. That is easier than a real directional sound field.

> **Caveats.** (1) The detector thresholds were tuned on the same 9 clips, and the margins are thin: Hovering 10 m peaks at 74 hits against 70 needed, and the ambient peaks at 63. (2) The team test windows come from the same recordings as the training windows (different time segments), so a whole-session holdout is still the real test. (3) All recordings so far are from a phone, not the MP34DT01-M PDM mics: expect domain shift (section 8). (4) The firmware has never been compiled or run on hardware; the bearing is untested outside simulation.

6.5 Expected real-world range (team estimate, not measured)

These figures are the team's expectation and have not been measured. Within ~30 m in quiet ambient, expect 80–90 % of flights to be detected within 3–5 s. Through the headband mics before a retrain, 50–80 %; after one field session and a retrain, 85–95 %. At ~50 m, 50–80 %; at 100 m, likely under 30 %. False alarms: about 0–2 per hour in quiet outdoor ambient, several per hour near mowers, chainsaws and vacuums. Bearing: typically 5–20° when the drone is loud; untested on hardware.

7. Firmware (`firmware/esp32s3/headband.ino`)

Status: written against the ESP-IDF 5.x headers and docs and the mic datasheet; not yet compiled on hardware. The board is an ESP32-S3-DevKitC-1 on the Arduino-ESP32 3.x core (ESP-IDF 5.1–5.5 underneath). From the DSP core down, it is the same code that passes `compare_core.py`.

7.1 What the sketch does

Task (core)	Job
`dsp_task` (core 0, prio 5)	PDM capture: I2S0 in PDM RX mode, 2 data lines, one 3.072 MHz clock, so all three channels are sample-aligned by construction. Reads 1024-sample chunks and runs spectra, log-mel, detector and bearing every chunk. At boot it prints `# pdm chunk_us=… (expect 21333)`.
`ml_task` (core 1, prio 2)	Classifier every 500 ms (TFLM, 60 KB arena) when `USE_TFLM 1`. With `USE_TFLM 0` (the current default), p = 1 and only the harmonic path decides.
`alert_task` (core 1, prio 3)	Buzzer chirp at detection onset. Motor PWM (200 Hz, 8-bit, duty capped at 0.6 for 3 V motors on the 5 V rail); pulse rate 1–5 Hz from strength. LED on while DETECTED, blinking otherwise.
`telemetry_task` (core 1, prio 1)	The only writer to `Serial`. In telemetry mode it sends one CSV `T` line per frame; in raw mode it sends binary 3-channel audio packets; it reads single-byte host commands (section 7.5).

7.2 Wiring (from the sketch header; jumper wires, no PCB)

Mic breakout pin	mic L	mic R	mic B	ESP32-S3-DevKitC-1
3V	3V3	3V3	3V3	3V3 pin
GND	GND	GND	GND	GND pin
CLK	GPIO 4	GPIO 4	GPIO 4	GPIO 4 = PDM_CLK (output, 3.072 MHz)
DAT	GPIO 5	GPIO 5	GPIO 6	GPIO 5 = PDM_DATA0 (L + R), GPIO 6 = PDM_DATA1 (B)
SEL	GND	3V3	GND	(strap on the breakout, not a GPIO)

Output	Motor L	Motor R	Motor B	Buzzer	Status LED
GPIO	7	8	13	9	48

Mics: Adafruit PDM MEMS breakouts (product 3492 / 4346, ST MP34DT01-M, 3.3 V, PDM clock 1–3.25 MHz). Geometry: L (–0.10, 0), R (+0.10, 0), B (0, –0.12) m. L and R share DAT0 as a stereo PDM pair: SEL low drives one clock phase and SEL high the other, so the two tri-state alternately. B is alone on DAT1. Keep the CLK wire short (it fans out to three mics at 3 MHz) and route it away from the motor wires. Pins avoided: strapping pins 0/3/45/46, USB 19/20, octal PSRAM 35–37.

7.3 Build notes

- Arduino IDE or arduino-cli with the **Arduino-ESP32 3.x** core; board ESP32-S3-DevKitC-1. Open `firmware/esp32s3/` as the sketch folder. `src/` holds symlinks to `../core`; on a checkout without symlinks, copy the three files in instead.
- **USB CDC on boot: enabled**, so that `Serial` is the native USB port. Telemetry and raw-audio streaming go over it at 921600 baud.
- Classifier: set `#define USE_TFLM 1` and install **esp-tflite-micro** (Espressif's component) or the `TensorFlowLite_ESP32` Arduino library. The sketch includes `model_data.h` from the sketch folder.
- Optional ESP-DSP: replace `dsp::fft_*` with `dsps_fft2r_fc32` (about 5× faster; ~1.5 ms of the 21 ms frame budget).
- RAM (README): `dsp_core.cpp` static arrays 57 KB, plus the sketch's objects 80 KB, plus its buffers 69 KB (including the 25 KB raw-audio stream ring), plus the 60 KB TFLM arena when enabled.

7.4 Bench checks, in this order

1. **Slot / SEL mapping.** Tap each mic in turn with a fingernail and watch which channel spikes. If L/R/B land in the wrong place, fix `CH_SLOT[]`; no other code changes. Also confirm the boot line `chunk_us` is ~21333 µs. This validates the assumed 4-slot DMA frame; ~16000 would mean 3 slots.

2. **DC offset per channel.** Gain does not matter (the classifier normalises each window), but the S3 PDM RX has no hardware high-pass filter, so check each channel's mean.
3. **Clean audio at 3.072 MHz.** This is above the Adafruit guide's 1–3 MHz but inside the MP34DT01-M's 3.25 MHz maximum, and it comes from a fractional divider of the 160 MHz PLL. Record a few seconds with `scripts/record_device.py` and listen to it and look at its spectrum, checking for tones or crackle.

7.5 Haptic behaviour and serial protocol

Drone at	Motors
Left (–90°)	L
Back-left (–135°)	B + L (panned)
Ahead (0°)	L + R together (no front motor)
Overhead (conf < 0.5 for ~0.5 s)	All three at 1.0, pulsing together

The pulse rate is 1–5 Hz, rising with harmonic strength. `alert_task` computes the weights every 20 ms, so they can trail a frame by one alert period.

Direction	Content
Host → device	Single ASCII bytes. `T`: telemetry mode (the default); it reprints the `H` header line. `R`: raw mode, binary packets only. `S`: one `# status …` line (ignored in raw mode).
Device → host, telemetry	Lines starting with `#` are comments (including `# pdm chunk_us=…`). Then the header and one `T` line per frame (~47 lines/s): `H,frame,t_ms,f0,score,strength,flat,hits,ml,detected,theta,conf,overhead,hap_L,hap_R,hap_B`
Device → host, raw	One packet per chunk, little endian: `A5 5A`, then uint16 n_frames (= 1024), uint32 chunk_index, then 1024 × 3 int16 interleaved L,R,B (6152 bytes). A gap in chunk_index means chunks were dropped; the DSP never waits on `Serial`. Raw mode streams ~288 KB/s, which needs the S3's **native USB** port with USB CDC On Boot enabled. Over the UART bridge at 921600 baud most chunks are dropped.

8. Device bring-up and field session

From `README.md` ("Device bring-up, plug and play"), `TRAINING_PLAN.md` sections 4–5 and the help text of the three host tools. None of this has been run against a real board yet.

8.1 Bring-up

1. **Wire** the three mics and the motors as in 7.2.
2. **Flash** with Arduino-ESP32 3.x (plus esp-tflite-micro if `USE_TFLM 1`). Use the native USB port with *USB CDC On Boot: Enabled*.
3. **Watch it run:** `scripts/telemetry.py --log run1.csv`. You should see ~47 lines/s and `# pdm chunk_us=` close to 21333. Plot the log afterwards with `--plot run1.csv --out run1.png`.
4. **Tap test:** run `scripts/record_device.py --monitor` and tap each mic in turn. The L, R and B columns should jump in that order; if they don't, fix `CH_SLOT[]` in the sketch. The same display shows DC offset and clipping (bench checks 7.4, steps 1–2).
5. **Audio quality:** record ~10 s (`record_device.py --seconds 10 --out /tmp/bench.wav`), then listen and look at the spectrum for tones or crackle (check 3). `run_file.py /tmp/bench.wav` also gives the bearing, because the file has 3 channels.
6. **Haptics:** walk a phone playing a drone clip around the head. Check L, B+L, B, B+R, R and L+R for ahead, then hold it overhead and check that all three buzz. The `hap_*` columns in the telemetry log show the same weights.

8.2 Field session: what to record (through the headband mics)

Recording	Command (one file per pass)	Notes
Drone passes at 10 / 25 / 50 / 100 m, behind a building, transits, overhead	`record_device.py --drone --holdout --seconds 120 --mono-check`	Target 15–20 min of drone audio in total. Put distance, condition and bearing in the file name (e.g. `mavic_25m_behind_building_45deg`). Trim to the flying segments; leave passing cars in.
Backgrounds: quiet ambient, traffic, people talking	`record_device.py --holdout --seconds 300`	Target ~40 min.
Our own motors cycling	same, with the motors running	Headband vibration. If it triggers detections, the fix is mechanical first (foam under the mic pods).
Lawn mower (also chainsaw, generator if available)	same	These share the comb structure. A mower recording is "worth more than another hour of drone audio" (TRAINING_PLAN).

Keep one complete session as `*_holdout.*`: the loader puts those files entirely in the test split, and `retune_holdout.py` uses them by default. Record a second, non-holdout session for training if there is time. Nothing in `data/team/` is committed.

8.3 Retune and retrain

1. The recordings land in `data/team/drone/` and `data/team/background/` (or copy them there).
2. **Thresholds:** run `.venv/bin/python scripts/retune_holdout.py`. Read the table: drone clips detected, background clips with any detection (must be 0) and ESC-50 frames. Then `--apply` writes the winner into `config.py` and regenerates `dsp_config.h`. To tune by hand instead, compare the score distributions that `run_file.py` prints for a drone file and a background file, and put `HARM_THRESH_DB` / `ML_THRESH` in the gap. Tune on the held-out session, never on training data.
3. **Classifier** (if it is what fails): retrain from scratch with `scripts/train.py --epochs 30 --n-synth 60`. With real positives present, synthetic clips stay a minority. In `train_report.json`, `team:drone` recall is the headline number, and the `team:background` FPR (now including our motors) is the number that decides the demo.
4. `scripts/export_c_headers.py`, copy `artifacts/model_data.h` into `firmware/esp32s3/`, reflash. Budget: about 20 minutes of compute, an hour including labelling.

8.4 What to watch for (TRAINING_PLAN section 5)

- **Domain shift** (phone vs MP34DT01-M frequency response and noise) is the main failure mode. Window normalisation removes gain differences but not differences in spectral shape.
- **Far drones** are faint and low-passed by the air, so check recall on the `*_100m*` files.
- **Over-trusting accuracy:** keep the splits by source. A random window split would report 99 % and mean nothing.

9. Known issues and next steps

Known issues

Issue	Detail / workaround
Firmware never compiled	The serial protocol and the host tools (`telemetry.py`, `record_device.py`) have never talked to a real board. The PDM multi-line DMA interleaving is not documented by Espressif and is assumed (`TODO(bench)` in the sketch). Run the bench checks in 7.4 before anything else.
Thresholds tuned on the test clips	9/9 detection is optimistic, and margins are thin (74 vs 70 hits on Hovering 10 m; ambient peaks at 63). Re-tune on a held-out session (section 8).

Domain shift	All training audio is phone or public data, not the MP34DT01-M mics. Expect an optimistic classifier until the field session is in the training data.
Bare `pytest -q` fails with ESC-50 present	It also collects `data/ESC-50/tests` (needs `librosa`). Use `pytest -q tests`.
`get_dads.py` default ≠ shipped data	The default `--shards 0,38` differs from the 6 shards used (`0,5,12,20,28,36`). Pass them explicitly to reproduce the shipped model.
Shipped model uses old synthetic data	A retrain with the newer `simulate.py` defaults scored worse (int8 acc 92.9 %, recall 84.8 %, FPR 4.7 %), so the shipped model is the older one. Next retrain: try `--n-synth 0-30`.
`artifacts/example_drone_3ch.wav` never reaches DETECTED	Observed while writing this document: `run_file.py` on it gives a steady bearing of about −120° but 0 % DETECTED. Its flatness of 0.02–0.06 is below the 0.17 floor, so it looks like it predates the flatness floor. No script uses it. For a 3-channel test file, use `compare_core.py`'s default (it writes `artifacts/compare_input.wav`) or `demo_direction.py`.
`train.py` overwrites committed artifacts	Use `--out` for experiments. Copy `model_data.h` into `firmware/esp32s3/` by hand after a real retrain.
Hot spots on the ESP32	The radix-2 FFT (~6 per frame) and the spectral baseline (3 × 1025 × 31 double adds per frame, on a core without a double FPU). Use esp-dsp and check frame timing.
Legacy `main` scripts	`detection_model.py` and `sig_proc_tri.py` need torch, datasets and scikit-learn, which are not in `requirements.txt`.

Next steps, in order

1. Build the sketch, flash it and pass the bench checks (7.4). Then set `USE_TFLM 1` and confirm inference time (estimated 60–150 ms per window).
2. Field session through the headband mics (section 8), including a lawn mower and the motors cycling.
3. Retrain, re-tune `HARM_THRESH_DB` / `ML_THRESH` on the held-out session, run `export_c_headers.py`, copy `model_data.h` and reflash.
4. Measure the real bearing error against known positions, and the false alarms per hour outdoors.
5. Stretch goals (TRAINING_PLAN section 6): approach/recede output from f0 Doppler drift; multiclass if a fixed-wing is recorded; distillation if recall at 100 m is the bottleneck.